

Day 45: Full revision

1. Number of Islands

👉 Problem

2D grid me 1 = land, 0 = water

Find how many islands (connected 1s)

👉 Idea (DFS)

Har cell check karo

Agar 1 mila → DFS chalao

Us island ko mark karo (visited)

Count++

✅ Code (Java)

```
class Islands {
    int n, m;

    public int numIslands(char[][] grid) {
        n = grid.length;
        m = grid[0].length;
        int count = 0;

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (grid[i][j] == '1') {
                    dfs(grid, i, j);
                    count++;
                }
            }
        }
        return count;
    }

    void dfs(char[][] grid, int i, int j) {
        if (i < 0 || j < 0 || i >= n || j >= m || grid[i][j] == '0')
            return;

        grid[i][j] = '0'; // mark visited

        dfs(grid, i+1, j);
        dfs(grid, i-1, j);
        dfs(grid, i, j+1);
        dfs(grid, i, j-1);
    }
}
```

🔥 Dry Run

1 1 0

0 1 0

1 0 1

Islands:

Top left group

Bottom left

Bottom right

👉 Answer = 3

● 2. Clone Graph

👉 Problem

Graph diya hai → exact copy banana hai

👉 Idea

Har node ko copy karo

Map use karo (original → clone)

DFS se traverse karo

✓ Code

```
class Node {
    public int val;
    public List<Node> neighbors;

    public Node(int val) {
        this.val = val;
        neighbors = new ArrayList<>();
    }
}

class CloneGraph {
    HashMap<Node, Node> map = new HashMap<>();

    public Node cloneGraph(Node node) {
        if (node == null) return null;

        if (map.containsKey(node))
            return map.get(node);

        Node clone = new Node(node.val);
        map.put(node, clone);

        for (Node neighbor : node.neighbors) {
            clone.neighbors.add(cloneGraph(neighbor));
        }

        return clone;
    }
}
```

🔥 Key Point

👉 Without map → infinite loop (cycle)

🟢 3. Detect Cycle (DFS)

👉 Undirected Graph

Idea:

Parent track karo

Agar visited node mila aur parent nahi hai → cycle

✓ Code

```
1. boolean dfs(int node, int parent, boolean[] visited) {
    visited[node] = true;

    for (int neighbor : adj.get(node)) {
        if (!visited[neighbor]) {
            if (dfs(neighbor, node, visited))
                return true;
        } else if (neighbor != parent) {
            return true;
        }
    }
    return false;
}
2.
```

👉 Directed Graph (Important 🔥)

Idea:

Recursion stack use karo

✓ Code

```
boolean dfs(int node, boolean[] visited, boolean[] recStack) {
    visited[node] = true;
    recStack[node] = true;

    for (int neighbor : adj.get(node)) {
        if (!visited[neighbor] && dfs(neighbor, visited, recStack))
            return true;
        else if (recStack[neighbor])
            return true;
    }

    recStack[node] = false;
    return false;
}
```

● 4. Rat in a Maze (Backtracking)

👉 Problem

Rat (0,0) se (n-1,n-1) tak jana hai

👉 Moves:

D (down)

R (right)

U (up)

L (left)

✓ Code

```
class Maze {
    int n;

    void solve(int[][] maze, int i, int j, String path) {
        if (i == n-1 && j == n-1) {
            System.out.println(path);
            return;
        }

        if (i < 0 || j < 0 || i >= n || j >= n || maze[i][j] == 0)
            return;

        maze[i][j] = 0; // mark visited

        solve(maze, i+1, j, path + "D");
        solve(maze, i, j+1, path + "R");
        solve(maze, i-1, j, path + "U");
        solve(maze, i, j-1, path + "L");

        maze[i][j] = 1; // backtrack
    }
}
```

🔥 Key Insight

👉 DFS + Backtracking combo

5. Surrounded Regions

👉 Problem

'O' ko capture karo agar wo boundary se connected nahi hai

👉 Idea (Important 🔥)

Boundary se DFS chalao

Safe 'O' mark karo (e.g. '#')

Baaki 'O' → 'X'

✅ Code

```
class Surrounded {
    int n, m;

    public void solve(char[][] board) {
        n = board.length;
        m = board[0].length;

        // boundary DFS
        for (int i = 0; i < n; i++) {
            dfs(board, i, 0);
            dfs(board, i, m-1);
        }

        for (int j = 0; j < m; j++) {
            dfs(board, 0, j);
            dfs(board, n-1, j);
        }

        // convert
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (board[i][j] == 'O')
                    board[i][j] = 'X';
                else if (board[i][j] == '#')
                    board[i][j] = 'O';
            }
        }
    }

    void dfs(char[][] board, int i, int j) {
        if (i < 0 || j < 0 || i >= n || j >= m || board[i][j] != 'O')
            return;

        board[i][j] = '#';

        dfs(board, i+1, j);
        dfs(board, i-1, j);
        dfs(board, i, j+1);
        dfs(board, i, j-1);
    }
}
```

 Final Summary (VERY IMPORTANT)

Problem	Pattern
Number of Islands	DFS on grid
Clone Graph	DFS + HashMap
Detect Cycle	DFS + parent / stack
Rat in Maze	Backtracking
Surrounded Regions	Boundary DFS